Web-Based Real Estate Price Prediction System

Software Requirements Specification

Version 1.0

Date: December 15, 2025

Prepared By: Khaleeq Ur Rehman
Roll Number: F22BDOCS1M01089
Program: Computer Science
Supervisor: Mr. Abdullah Soomro
Institution: Islamia University Bahawalpur

Revision History

Date	Version	Description	Author	Comments
Dec 15, 2025	1.0	Initial SRS Document	Khaleeq Ur Rehman	First Revision

---

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Khaleeq Ur Rehman	Student, FYP Part 1	
	Mr. Abdullah Soomro	Supervisor, CS Dept	

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Scope

- 1.3 Definitions, Acronyms, and Abbreviations
- 1.4 References
- 1.5 Overview

2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Characteristics
    - 2.4 General Constraints
    - 2.5 Assumptions and Dependencies
3. Specific Requirements
    - 3.1 External Interface Requirements
    - 3.2 Functional Requirements
    - 3.3 Use Cases
    - 3.4 Classes/Objects
    - 3.5 Non-Functional Requirements
    - 3.6 Design Constraints
    - 3.7 Logical Database Requirements
4. Analysis Models
    - 4.1 Data Flow Diagrams
    - 4.2 Sequence Diagrams
5. Appendices

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes all functional and non-functional requirements for the Web-Based Real Estate Price Prediction System. This document is intended for:

- The development team (student implementing the project)
- Project supervisor and evaluation committee
- Future developers who may maintain or enhance the system

The SRS provides complete information needed to design and implement the software product.

1.2 Scope

Product Name: Web-Based Real Estate Price Prediction System

Product Description:

This system is a web-based application that predicts real estate prices using machine learning. It allows users to upload property datasets, view statistical analysis, and get price predictions for new properties.

Key Features:

- CSV file upload and validation
- Statistical analysis of property data
- Interactive data visualizations
- Price prediction using Linear Regression
- Results export functionality

Goals:

- Achieve 70-80% prediction accuracy using Linear Regression
- Process datasets with 500-2000 records
- Provide predictions within 2-3 seconds
- Support multiple property features (Location, Area, Bedrooms, Bathrooms, Age)
- Generate interactive charts for data insights

Out of Scope for Part 1:

- User authentication and login system
- Database integration (using CSV files only)
- Mobile application
- REST API
- Real-time data updates
- Advanced ML algorithms beyond Linear Regression
- Cloud deployment
- Payment integration

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
API	Application Programming Interface
CSV	Comma-Separated Values file format
DFD	Data Flow Diagram
FYP	Final Year Project
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
MAE	Mean Absolute Error - average prediction error
ML	Machine Learning
PKR	Pakistani Rupee
R^2 Score	Coefficient of Determination - model accuracy metric (0 to 1)
SRS	Software Requirements Specification
UI	User Interface
Property Features	Attributes like location, size, bedrooms, bathrooms, age
Training Data	Historical property data used to train ML model
Prediction	Estimated property price generated by ML model

1.4 References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
2. Flask Documentation - https://flask.palletsprojects.com/
3. Scikit-learn Documentation - https://scikit-learn.org/
4. Chart.js Documentation - https://www.chartjs.org/
5. Pandas Documentation - https://pandas.pydata.org/
6. Project Proposal: "Web-Based Real Estate Price Prediction System" (Approved November 26, 2025)

1.5 Overview

This SRS document is organized into five main sections:

- **Section 2** provides general description including product perspective, functions, user characteristics, and constraints
- **Section 3** details all specific functional and non-functional requirements
- **Section 4** presents analysis models including DFDs and sequence diagrams
- **Section 5** contains appendices with supplementary information

---

2. Overall Description

2.1 Product Perspective

The Real Estate Price Prediction System is a **standalone web application** with no integration to existing systems. It uses a simple client-server architecture:

System Components:

- **Client Side:** Web browser running HTML/CSS/JavaScript interface
- **Server Side:** Python Flask application handling requests and processing
- **Data Storage:** CSV file-based storage (no database in Part 1)
- **ML Engine:** Scikit-learn library with Linear Regression
- **Visualization:** Chart.js library for interactive charts

System Boundaries:

- **In Scope:** Data upload, validation, analysis, visualization, ML prediction, export
- **Out of Scope:** User authentication, database, mobile app, API, advanced ML algorithms

Major Interfaces:

- User web interface
- File system for CSV storage
- Trained ML model files

2.2 Product Functions

The system provides the following main functions:

1. Data Upload and Management

- Accept CSV file uploads through web interface
- Validate file format and structure
- Preview uploaded data
- Store data for analysis

2. Statistical Analysis

- Calculate descriptive statistics (mean, median, std, min, max)
- Perform location-wise price analysis
- Calculate correlations between features
- Analyze property type distribution

3. Data Visualization

- Bar charts for average prices by location
- Scatter plots for price vs. area relationships
- Pie charts for property type distribution
- Statistics dashboard with key metrics

4. Price Prediction

- Train Linear Regression model on uploaded data
- Accept user input for property features
- Generate price predictions
- Display confidence level and price range

5. Export Results

- Export analyzed data to CSV
- Download prediction results

2.3 User Characteristics

Primary Users: Property Buyers and Sellers

- **Technical Expertise:** Low to Medium (basic computer skills)
- **Domain Knowledge:** Basic understanding of property features
- **Usage Frequency:** Occasional (during property search/sale)
- **Key Needs:** Quick price estimates, market insights
- **Age Range:** 25-60 years
- **Education:** High school to postgraduate

Secondary Users: Real Estate Agents

- **Technical Expertise:** Medium
- **Domain Knowledge:** High (professional knowledge)
- **Usage Frequency:** Regular (weekly)
- **Key Needs:** Quick valuations, market analysis
- **Age Range:** 25-55 years
- **Education:** Bachelor's degree or higher

General User Requirements:

- Can navigate web browsers
- Understand basic property terminology
- Can upload files from computer
- Can interpret basic charts and statistics
- No programming knowledge required

2.4 General Constraints

Technical Constraints:

- Must run on modern web browsers (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+)
- Server requires Python 3.8 or higher
- Limited to CSV file format for data upload
- File size limited to 10MB per upload

- No concurrent multi-user support in Part 1
- Single-threaded processing

Development Constraints:

- Academic project timeline: 17 weeks for Part 1
- No budget for paid services or APIs
- Must use open-source technologies only
- Single developer (student) implementation
- Code must be documented for academic evaluation

Regulatory Constraints:

- Must comply with Pakistan Electronic Crimes Act 2016
- Cannot guarantee legal accuracy of price predictions
- User must accept disclaimer before using predictions

2.5 Assumptions and Dependencies

Assumptions:

1. Users have access to modern web browsers
2. Users have basic computer literacy
3. Users understand property terminology
4. Users will provide accurate input data
5. Server has sufficient storage (minimum 1GB available)
6. Server has adequate processing power (minimum 2GB RAM)
7. Property data is accurate and up-to-date
8. Dataset contains minimum 500 records for training

Dependencies:

1. **Software Dependencies:**
    - Python 3.8+ (core runtime)
    - Flask 2.3+ (web framework)

- Pandas 2.0+ (data processing)
- NumPy 1.24+ (numerical computations)
- Scikit-learn 1.3+ (machine learning)
- Chart.js 4.0+ (visualizations via CDN)

2. **Data Dependencies:**
    - Availability of quality property datasets
    - Consistent data format across datasets
3. **Hardware Dependencies:**
    - Computer to host Flask application
    - Minimum 2GB RAM for processing
    - Minimum 10GB storage space

---

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

UI-1: Home Page

- Header with system logo and title
- Navigation menu (Home, Upload, Dashboard, Predict)
- Welcome message and brief system description
- Quick action buttons

UI-2: Upload Page

- File upload area with drag-drop support
- File requirements information
- Upload button
- Data preview section (first 20 rows)
- Success/error messages

UI-3: Dashboard Page

- Statistics cards displaying:
  - Total properties
  - Average price
  - Minimum price
  - Maximum price
- Four charts in 2x2 grid:
  - Bar chart: Average price by location
  - Scatter plot: Price vs. area
  - Pie chart: Property type distribution
  - Additional analysis chart
- Export data button

UI-4: Prediction Page

- Input form with fields:
  - Location (dropdown)
  - Area in square feet (number input)
  - Number of bedrooms (dropdown)
  - Number of bathrooms (dropdown)
  - Property age in years (number input)
- Predict button
- Results panel showing:
  - Predicted price
  - Confidence level
  - Price range (±10%)

UI-5: Error Pages

- Clear error messages
- Suggested corrective actions
- Navigation back to main pages

3.1.2 Hardware Interfaces

The system has no direct hardware interface requirements. All hardware interaction is handled through the operating system and web browser.

Server Hardware Requirements:

- Minimum: Intel Core i3, 4GB RAM, 10GB storage
- Recommended: Intel Core i5, 8GB RAM, 20GB SSD

Client Hardware Requirements:

- Any device with web browser
- Minimum: 2GB RAM, 1024x768 display
- Recommended: 4GB RAM, 1920x1080 display

3.1.3 Software Interfaces

SI-1: Flask Framework (v2.3+)

- Purpose: Web application framework
- Interface: Python API
- Functions: Routing, request handling, template rendering

SI-2: Pandas Library (v2.0+)

- Purpose: Data manipulation and analysis
- Interface: Python API
- Functions: read_csv(), describe(), groupby(), to_csv()

SI-3: Scikit-learn Library (v1.3+)

- Purpose: Machine learning
- Interface: Python API
- Functions: LinearRegression(), train_test_split(), fit(), predict()

SI-4: Chart.js Library (v4.0+)

- Purpose: Data visualization

- Interface: JavaScript API via CDN
- Functions: Bar, line, scatter, and pie charts

3.1.4 Communications Interfaces

HTTP Protocol:

- Protocol: HTTP/1.1
- Port: 5000 (development)
- Methods: GET (retrieve pages), POST (submit forms, upload files)
- Data Format: JSON for AJAX requests, multipart/form-data for file uploads

3.2 Functional Requirements

FR-1: Data Upload Module

FR-1.1: File Upload

- **Priority:** High
- **Description:** System shall provide web interface for uploading CSV files
- **Input:** CSV file (max 10MB)
- **Processing:** Validate file extension, check size, store temporarily
- **Output:** Success/error message
- **Validation:** Only .csv extension, size ≤ 10MB

FR-1.2: Data Validation

- **Priority:** High
- **Description:** System shall validate uploaded CSV structure
- **Input:** Uploaded CSV file
- **Processing:** Check required columns (Location, Price, Area, Bedrooms, Bathrooms), verify data types, check minimum 100 records
- **Output:** Validation success or detailed error messages
- **Required Columns:** Location (string), Price (numeric >0), Area (numeric 100-10000), Bedrooms (integer 1-10), Bathrooms (integer 1-8)

FR-1.3: Data Preview

- **Priority:** Medium
- **Description:** System shall display preview of uploaded data
- **Input:** Validated CSV file
- **Processing:** Read first 20 rows using Pandas
- **Output:** HTML table showing first 20 records

FR-2: Statistical Analysis Module

FR-2.1: Descriptive Statistics

- **Priority:** High
- **Description:** System shall calculate comprehensive statistics
- **Input:** Property dataset
- **Processing:** Calculate mean, median, std, min, max, quartiles for Price column
- **Output:** Dictionary with statistical values

FR-2.2: Location Analysis

- **Priority:** High
- **Description:** System shall analyze prices by location
- **Input:** Property dataset
- **Processing:** Group by Location, calculate average/min/max price per location, count properties
- **Output:** Location-wise analysis results

FR-2.3: Correlation Analysis

- **Priority:** Medium
- **Description:** System shall calculate feature correlations
- **Input:** Property dataset
- **Processing:** Calculate correlations between Price and (Area, Bedrooms, Bathrooms, Age)
- **Output:** Correlation matrix

FR-3: Visualization Module

FR-3.1: Bar Chart Generation

- **Priority:** High
- **Description:** System shall generate bar chart for location prices
- **Input:** Location-wise analysis data
- **Processing:** Prepare Chart.js format with labels and datasets
- **Output:** JSON data for Chart.js bar chart

FR-3.2: Scatter Plot Generation

- **Priority:** High
- **Description:** System shall generate scatter plot for Price vs Area
- **Input:** Property dataset with Area and Price columns
- **Processing:** Extract x (Area) and y (Price) coordinates
- **Output:** JSON data for Chart.js scatter plot

FR-3.3: Pie Chart Generation

- **Priority:** Medium
- **Description:** System shall generate pie chart for property types
- **Input:** Property type distribution data
- **Processing:** Create segments with labels and percentages
- **Output:** JSON data for Chart.js pie chart

FR-4: Machine Learning Module

FR-4.1: Model Training

- **Priority:** High
- **Description:** System shall train Linear Regression model
- **Input:** Property dataset (minimum 500 records)
- **Processing:**
    - Split data 80/20 (training/testing)
    - Encode categorical variables (Location)
    - Extract features: Location, Area, Bedrooms, Bathrooms, Age
    - Train LinearRegression model

    - Calculate R^2 score and MAE
- **Output:** Trained model, encoder, performance metrics
- **Success Criteria:** R^2 score ≥ 0.50

FR-4.2: Model Persistence

- **Priority:** High
- **Description:** System shall save and load trained model
- **Input:** Trained model object
- **Processing:** Use pickle to serialize/deserialize model and encoder
- **Output:** model.pkl and encoder.pkl files

FR-4.3: Price Prediction

- **Priority:** High
- **Description:** System shall predict property prices
- **Input:** Property features (Location, Area, Bedrooms, Bathrooms, Age)
- **Processing:**
    - Validate inputs
    - Encode categorical features
    - Apply loaded model
    - Calculate prediction
- **Output:** Predicted price, confidence level, price range (±10%)
- **Response Time:** < 1 second

FR-5: Export Module

FR-5.1: Data Export

- **Priority:** Medium
- **Description:** System shall export analyzed data
- **Input:** Processed dataframe
- **Processing:** Convert to CSV, add timestamp to filename
- **Output:** CSV file download

3.3 Use Cases

Use Case 1: Upload Property Dataset

Use Case ID: UC-1
Actors: User (Buyer/Seller/Agent)
Preconditions: User has CSV file with property data
Postconditions: CSV file uploaded, validated, and available for analysis

Main Flow:

1. User navigates to Upload page
2. System displays file upload interface
3. User selects CSV file
4. User clicks Upload button
5. System validates file format and structure
6. System displays data preview (first 20 rows)
7. System saves file and displays success message

Alternative Flows:

- **5a. Invalid Format:** System displays error "Please upload a CSV file"
- **5b. Missing Columns:** System displays error listing missing required columns
- **5c. File Too Large:** System displays error "File exceeds 10MB limit"

Frequency: Multiple times per user

Use Case 2: View Analytics Dashboard

Use Case ID: UC-2
Actors: User
Preconditions: Data has been uploaded
Postconditions: User views analysis and charts

Main Flow:

1. User clicks Dashboard in navigation
2. System loads dataset
3. System calculates statistics

4. System generates charts
5. System displays dashboard with statistics cards and 4 charts
6. User views and interacts with visualizations

Alternative Flows:

- **2a. No Data:** System displays "Please upload data first" message

Frequency: Multiple times per session

Use Case 3: Predict Property Price

Use Case ID: UC-3
Actors: User (Buyer/Seller)
Preconditions: ML model has been trained
Postconditions: Price prediction displayed

Main Flow:

1. User navigates to Predict page
2. User selects location from dropdown
3. User enters area, bedrooms, bathrooms, age
4. User clicks Predict Price button
5. System validates inputs
6. System loads ML model
7. System generates prediction
8. System displays predicted price, confidence level, and price range

Alternative Flows:

- **5a. Invalid Input:** System highlights invalid fields, displays error messages
- **6a. Model Not Found:** System displays "Model not trained yet, please upload data first"

Frequency: Multiple predictions per user

3.4 Classes/Objects

Class 1: PropertyDataset

Purpose: Manage property data loading, validation, and storage

Attributes:

- filename: String - Name of uploaded CSV file
- dataframe: DataFrame - Pandas DataFrame with property data
- upload_time: DateTime - Timestamp of upload
- record_count: Integer - Number of property records

Methods:

- load_csv(filepath: String) → Boolean: Load CSV into DataFrame
- validate_data() → Boolean: Validate structure and content
- get_preview(rows: Integer) → DataFrame: Get first N rows
- get_statistics() → Dictionary: Calculate descriptive statistics
- export_csv(filepath: String) → Boolean: Export data to CSV

Class 2: DataAnalyzer

Purpose: Perform statistical analysis on property data

Attributes:

- dataset: PropertyDataset - Reference to dataset
- statistics: Dictionary - Calculated statistics
- correlations: DataFrame - Correlation matrix

Methods:

- calculate_statistics() → Dictionary: Calculate all statistics
- location_analysis() → DataFrame: Analyze by location
- correlation_analysis() → DataFrame: Calculate correlations
- price_distribution() → Dictionary: Analyze price ranges

Class 3: DataVisualizer

Purpose: Generate charts and visualizations

Attributes:

- analyzer: DataAnalyzer - Reference to analyzer

- chart_config: Dictionary - Chart configuration settings

Methods:

- generate_bar_chart() → JSON: Bar chart data for Chart.js
- generate_scatter_plot() → JSON: Scatter plot data
- generate_pie_chart() → JSON: Pie chart data

Class 4: PredictionModel

Purpose: ML model management and predictions

Attributes:

- model: LinearRegression - Scikit-learn model
- encoder: LabelEncoder - Encoder for categorical features
- model_path: String - Path to saved model
- r2_score: Float - Model accuracy
- mae: Float - Mean Absolute Error

Methods:

- train(dataset: PropertyDataset) → Boolean: Train model
- save_model(path: String) → Boolean: Save to disk
- load_model(path: String) → Boolean: Load from disk
- predict(features: Array) → Float: Predict price
- evaluate() → Dictionary: Calculate performance metrics

Class 5: Property

Purpose: Represent individual property

Attributes:

- location: String - Property location
- area: Float - Area in square feet
- bedrooms: Integer - Number of bedrooms
- bathrooms: Integer - Number of bathrooms

- age: Integer - Property age in years
- property_type: String - Type (House, Apartment, Villa)
- price: Float - Property price

Methods:

- validate() → Boolean: Validate property data
- to_features() → Array: Convert to ML feature array

3.5 Non-Functional Requirements

NFR-1: Performance Requirements

- File upload processing: < 3 seconds for 1MB file
- Data analysis: < 5 seconds for 2000 records
- Chart rendering: < 2 seconds
- Price prediction: < 1 second
- Page load time: < 3 seconds

NFR-2: Reliability Requirements

- System uptime: 95% during development phase
- Data integrity: No data loss during processing
- Error recovery: System shall log errors and provide retry options
- Model predictions: Reproducible for same inputs

NFR-3: Usability Requirements

- New users complete first prediction within 5 minutes
- No training required for basic operations
- Clear instructions on every page
- Intuitive navigation
- Readable fonts (minimum 14px)
- High contrast text (4.5:1 ratio)

NFR-4: Security Requirements

- Accept only .csv file extensions
- Limit file size to 10MB
- Validate all user inputs
- Sanitize inputs to prevent XSS attacks
- Session data cleared after 24 hours
- No collection of personal data

NFR-5: Maintainability Requirements

- Code commented (minimum 20% comment ratio)
- Follow PEP 8 style guide for Python
- Functions limited to 50 lines
- Modular code structure
- Documentation for all functions

NFR-6: Portability Requirements

- Run on Windows 10+, macOS 10.15+, Ubuntu 20.04+
- Browser compatibility: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- No platform-specific dependencies

3.6 Design Constraints

1. Must use Python 3.8+ for backend development
2. Must use Flask framework for web application
3. Must use Linear Regression algorithm (no advanced ML in Part 1)
4. Must store data in CSV files (no database in Part 1)
5. Must use Chart.js for visualizations
6. Must use only open-source libraries
7. Code must follow PEP 8 Python style guide
8. All libraries must be free and open-source

3.7 Logical Database Requirements

Note: Part 1 uses file-based CSV storage, not a relational database.

CSV Data Structure:

Primary Data File: properties.csv

Column Name	Data Type	Constraints	Description
PropertyID	Integer	Unique	Property identifier
Location	String	NOT NULL	Property location
PropertyType	String	NOT NULL	House/Apartment/Villa
Area	Float	NOT NULL, >0	Area in square feet
Bedrooms	Integer	1-10	Number of bedrooms
Bathrooms	Integer	1-8	Number of bathrooms
Age	Integer	≥0	Property age in years
Price	Float	NOT NULL, >0	Property price

Model Storage:

- model.pkl: Trained Linear Regression model (5-50 KB)
- encoder.pkl: Label encoder for categorical variables (1-10 KB)
- metadata.json: Training date, dataset size, R^2 score

---

4. Analysis Models

4.1 Data Flow Diagrams

Level 0 DFD (Context Diagram)

```
[User] ---(Property Data)---> [Real Estate Price Prediction System] ---(Analysis Results)---> [User]
                                  |
                              (Trained Model)
                                  |
                              [File System]
```

Level 1 DFD

```
                [User]
                  |
        (CSV File)  |  (Property Features)
                  |
                  v
          +-------------------+
          |  Upload & Store  |
          +-------------------+
                  |
            (Validated Data)
                  |
        +------------+------------+
        |                        |
        v                        v
+------------------+     +------------------+
| Analyze & Visual |     |  Train & Predict |
+------------------+     +------------------+
        |                        |
    (Statistics,             (Predictions,
     Charts)                  Confidence)
        |                        |
        +------------+------------+
                  |
                  v
                [User]
```

4.2 Sequence Diagrams

Sequence Diagram: Upload and Analyze

```
User -> System: Select CSV file
System -> System: Validate file format
System -> FileSystem: Save CSV
System -> System: Parse and validate data
System -> System: Calculate statistics
System -> System: Generate charts
System -> User: Display dashboard
```

Sequence Diagram: Price Prediction

```
User -> System: Enter property details
System -> System: Validate inputs
System -> FileSystem: Load model.pkl
System -> Model: Encode features
System -> Model: Predict price
System -> System: Calculate confidence
System -> User: Display prediction + range
```

---

5. Appendices

Appendix A: Sample CSV Format

```csv
PropertyID,Location,PropertyType,Area,Bedrooms,Bathrooms,Age,Price
1,Gulberg,House,2000,4,3,5,15000000
2,DHA,Apartment,1200,3,2,2,8500000
3,Model Town,Villa,3000,5,4,1,25000000
```

Appendix B: Installation Instructions

Step 1: Install Python 3.8+

```bash
# Download from python.org
```

Step 2: Install Dependencies

```bash
```

```
pip install flask pandas numpy scikit-learn
```

Step 3: Run Application

```bash
python app.py
```

Step 4: Access System

- Open browser and navigate to: http://localhost:5000

Appendix C: Glossary

- **Linear Regression:** ML algorithm modeling linear relationship between features and price
- **Feature:** Input variable (e.g., area, bedrooms)
- **Target Variable:** Output being predicted (Price)
- **R^2 Score:** Model accuracy metric (0-1, higher is better)
- **MAE:** Average magnitude of prediction errors
- **CSV:** Plain text format for tabular data
- **DataFrame:** Pandas 2D data structure

Appendix D: Testing Checklist

Functional Testing:

- [ ] File upload with valid CSV
- [ ] File upload with invalid format
- [ ] File upload exceeding size limit
- [ ] Data validation with missing columns
- [ ] Statistics calculation accuracy
- [ ] Chart rendering
- [ ] Model training
- [ ] Price prediction accuracy
- [ ] Export functionality

Non-Functional Testing:

- [ ] Upload speed (< 3 seconds for 1MB)

- [ ] Analysis time (< 5 seconds for 1000 records)
- [ ] Prediction time (< 1 second)
- [ ] Browser compatibility
- [ ] Error handling
- [ ] Input validation

Document Certification

Student Declaration:

I, Khaleeq Ur Rehman, certify that:

1. This SRS accurately represents the Real Estate Price Prediction System
2. All requirements are documented to the best of my knowledge
3. I commit to implementing these requirements as specified
4. I will seek supervisor guidance when needed

Student Signature: _____
Date: _____

Supervisor Approval:

Requirements are:

- [ ] Clear and complete
- [ ] Technically feasible
- [ ] Appropriately scoped for FYP Part 1
- [ ] Following proper standards

Comments: _____

Supervisor Signature: _____
Date: _____

Status: [ ] Approved [ ] Needs Revision [ ] Rejected

End of Software Requirements Specification

Document Information:

- Total Pages: ~30 pages

- Word Count: ~6,000 words
- Revision: 1.0
- Status: Ready for Review

© 2025 Khaleeq Ur Rehman. All Rights Reserved.

This document is prepared as part of Final Year Project Part 1 requirements and is subject to academic integrity policies of Islamia University Bahawalpur.